Attention 加速器 IP 设计与验证

设计报告

2026 年 10 月 12 日

报告人：叶梓龙

1 公式理解

1.1 Attention 与 softmax

对于单个 Query Q 与 S 个 Key/Value 对，注意力输出为各 Value 的加权和：

$$O = \sum_{j=1}^{S} \mathrm{softmax}(\mathrm{score}_j) \cdot V_j, \qquad \mathrm{score}_j = Q \cdot K_j = \sum_{k=1}^{d} Q_k \, K_{j,k}. \tag{1}$$

softmax 权重为 $\mathrm{weight}_j = \dfrac{e^{\mathrm{score}_j}}{\sum_{j'} e^{\mathrm{score}_{j'}}}$。直接对 score_j 取指数存在溢出风险，因此引入**数值稳定**处理：先减去最大值 $x_{\max} = \max_j \mathrm{score}_j$，由于分子分母同除 $e^{x_{\max}}$，结果不变：

$$\mathrm{weight}_j = \frac{e^{\mathrm{score}_j - x_{\max}}}{\sum_{j'} e^{\mathrm{score}_{j'} - x_{\max}}} = \frac{e^{\mathrm{diff}_j}}{L}, \qquad \mathrm{diff}_j = \mathrm{score}_j - x_{\max} \le 0. \tag{2}$$

令 $\mathrm{diff}_j \le 0$，$e^{\mathrm{diff}_j} \in (0, 1]$，$L = \sum_j e^{\mathrm{diff}_j}$。于是输出写为

$$O_i = \frac{\mathrm{ACC}_i}{L}, \qquad \mathrm{ACC}_i = \sum_j \mathrm{weight}_j \cdot V_{j,i}. \tag{3}$$

1.2 七步算法

综合以上，硬件实现遵循如下七步（软件 golden model 完全一致）：

1. $\mathrm{score}_j = Q \cdot K_j = \sum_k Q_k K_{j,k}$（$d$ 项点积）；
2. $x_{\max} = \max_j \mathrm{score}_j$；
3. $\mathrm{diff}_j = \mathrm{score}_j - x_{\max}$；
4. $\mathrm{weight}_j = e^{\mathrm{diff}_j}$ 的 Q0.4 定点值（查表）；

5. $L = \sum_j \text{weight}_j$；

6. $\text{ACC}_i = \sum_j \text{weight}_j \cdot V_{j,i}$（$d$ 路并行）；

7. $O_i = \text{round}(\text{ACC}_i / L)$（按符号四舍五入）。

1.3 定点化：Q0.4

经确认，小数保留四位是 **4 个二进制小数位**（Q0.4），而非十进制 4 位。由于在 verilog 中并不能直接表示小数，于是将已保留四位二进制小数的数字乘以 16，得到对应的整数作为 weight。即二进制整数 16 表示实数 1.0，缩放因子为 $2^4 = 16$。真实值与定点整数间的关系为

$$\text{weight}_{\text{定点}} = \text{round}\left(e^{\text{diff}} \times 16\right). \tag{4}$$

采用 Q0.4 的合理性在于：weight 本身 ≤ 1，用 5 bit（1 整数位 + 4 小数位）即可表示，且无需浮点乘法器。更重要的是，缩放因子 16 在输出归一化处**恰好约掉**——ACC_i 与 L 都携带同一倍率，故

$$O_i = \text{round}\left(\frac{\text{ACC}_i}{L}\right) \quad \text{与倍率无关} \tag{5}$$

2 指数查表

由于 $\text{diff}_j = \text{score}_j - x_{\max}$ 为整数，指数只需在少量离散点上查表。$e^{\text{diff}} \times 16$ 的取值如表 1 所示。

表 1: exp LUT（Q0.4，$\text{weight} = \text{round}(e^{\text{diff}} \times 16)$）

diff	0	−1	−2	−3	≤ -4
e^{diff}	1.0	0.3679	0.1353	0.0498	≤ 0.0183
$\times 16$	16	5.89	2.17	0.80	≤ 0.29
weight	16	6	2	1	0
（二进制小数）	1.0000	0.0110	0.0010	0.0001	0

可见在 diff=0/-1/-2/-3 在二进制的小数中存在数值，相应的在 weight 分别存为 16/6/2/1。

3 硬件架构

3.1 顶层接口

顶层模块 `attention_ip` 参数化 S、d，接口如下：

```
module attention_ip #(parameter S = 8, parameter d = 4)(
    input clk, rst_n, start,
    input signed [7:0] q_in, k_in, v_in,
```

```
    input q_valid, kv_valid,
    output reg data_ready,
    output reg signed [7:0] out_data,
    output reg out_valid, done,
    output reg [3:0] state_now,//当前状态输出
    output reg [S-1:0] key//加分项D要求的输出
);
```

Listing 1: 顶层模块接口

输入 $Q/K/V$ 分时复用三条 8-bit 数据线：Q 在 q_valid 拉高时连续 d 拍灌入；K/V 在 kv_valid 拉高时连续 $S \times d$ 拍灌入。输出 O 在 OUTPUT 状态逐个周期给出，随后 $done = 1$。

3.2 状态机

采用九状态状态机，状态编码、职责与跳转条件如表 2 所示。

表 2: 状态机定义

状态	编码	职责	跳转条件
IDLE	0	等待启动	$start$
LOAD_Q	1	载入 Q（d 拍）	$cnt_q = d-1$
LOAD_KV	2	载入 K/V（$S \times d$ 拍）	$cnt_kvi = d-1 \wedge cnt_kvj = S-1$
DOT	3	点积得 score_j	$cnt_sci = d-1 \wedge cnt_scj = S-1$
MAX	4	求 $x_{\max}$ 与 diff_j	MAX_finish
EXP	5	查表得 weight_j	EXP_finish
UPDATE	6	累加 L 与 d 路 ACC	$UPDATE_finish$
NORM	7	$O_i = \text{round}(\text{ACC}_i/L)$	$NORM_finish$
OUTPUT	8	逐拍输出 O，置 $done$	$cnt_out = d-1$

3.3 位宽推导

位宽推导遵循两条基本原则：**乘法 $N \times M$ 得 $(N+M)$ bit；累加 K 项在单次位宽基础上加 $\lceil \log_2 K \rceil$ bit**。由此定义 localparam（$W = 8$ 为数据位宽，$Sw = \lceil \log_2 S \rceil = 3$，$dw = \lceil \log_2 d \rceil = 2$）：

其中 L 的最大值为 $S \times 16 = 128$（包含四位小数），故 $L_W = 8$；weight（5 bit，最大 16）扩展为 6-bit 有符号后与 8-bit 的 V 相乘得 14 bit，再累加 $S = 8$ 项加 3 bit 得 17 bit。

据此，各主要寄存器的位宽与含义汇总如表 3。表中深度指数组元素个数；打包指把 d（或 S）路数据拼成一条宽信号，取其中一路用带偏移的位选 `[W*k +: W]`。

几点说明：

- `q_reg`、`O` 与数据输入同宽（8-bit 有符号），因为 $O_i = \text{round}(\text{ACC}_i/L)$ 是加权平均，将小数 16 位消去，结果量级落在 V 的取值范围内。

表 3: 主要寄存器位宽与含义

信号	位宽	深度	含义
q_reg	8	$d = 4$	Q 的 d 个元素（W）
k_reg	32 ($= d \times W$)	$S = 8$	K 矩阵，每 token 打包 d 维
v_reg	32 ($= d \times W$)	$S = 8$	V 矩阵，每 token 打包 d 维
score	16	——	单次 $q \times k$ 乘积（$SCORE_W = 2W$）
score_reg	18	$S = 8$	score_j，d 项点积累加（$SCOREREG_W = 2W + dw$）
max_0	18	——	$x_{\max}$ 寄存器（$SCOREREG_W$）
weight	5	$S = 8$	$\text{weight}_j = e^{\text{diff}_j}$ 的 Q0.4（1 整数位 + 4 小数位）
max_weight	5	——	weight 的最大值，稀疏阈值用
L	8	——	$\sum_j \text{weight}_j$（$L_W = \lceil \log_2(S \times 16 + 1) \rceil$）
ACC	56 ($= d \times ACC_W$)	$S = 8$	每个 token 的 d 路 weight $\times V$ 累加（$ACC_W = 6 + W$
ACC_reg	68 ($= d \times ACCREG_W$)	——	d 路最终 ACC_i，对 S 个 token 求和（$ACCREG_W =$
O	8	$d = 4$	输出 O_i（与 V 同数据位宽 W）

- k_reg/v_reg 意思位将 k1,k2... 打包 $d \times W = 32$ bit，即一个 token 的 4 个维度连续存放，读写用 `[W*k +: W]` 按 $k = 0 \cdots 3$ 切片。
- ACC 每个元素 56 bit 内分 4 条 14-bit 通道（对应 4 个输出维度），ACC_reg 则是 4 条 17-bit 通道打包的 68 bit；二者都比单通道位宽 × 通道数，是为了并行累加 4 路而不串扰。
- score（16 bit）是**单拍**的 $q \times k$ 积，score_reg（18 bit）是**累计** d **次**后的完整点积，中间不截断，避免溢出。

4 关键模块实现

4.1 数据装载（LOAD_Q / LOAD_KV）

Q 存入 $q_reg[0 : d-1]$（8-bit 有符号）；K/V 存入 $k_reg/v_reg[0 : S-1]$，每个 token 打包为 $d \times W$ 位，用带偏移的位选 `[W*cnt_kvi +: W]` 写入对应维度。计数器 cnt_q、cnt_kvi/cnt_kvj 分别记录维度与 token 下标。

4.2 点积（DOT）

每个周期做一次 $q_reg[cnt_sci] \times k_reg[cnt_scj]$，累加进 score_reg[$cnt_scj$]。循环 d 次得到一个 score_j，循环 S 次得到全部 8 个 score。

4.3 求最大值与差分（MAX）

组合逻辑扫描 score_reg 求 $x_{\max} = \max$；差分 $\text{diff}_j = \text{score}_j - x_{\max}$ 同样组合算出，在 MAX 状态一拍寄存进 x_max0 数组。

4.4 指数查表（EXP）

对每个 diff_j 用 case 查表得 weight_j（表 1）：

```
case (x_max0[i])
    0: weight[i] <= 5'b10000; // 16 = 1.0000
    -1: weight[i] <= 5'b00110; // 6 = 0.0110
    -2: weight[i] <= 5'b00010; // 2 = 0.0010
    -3: weight[i] <= 5'b00001; // 1 = 0.0001
    default: weight[i] <= 5'b00000;
endcase
```

Listing 2: exp LUT 实现

4.5 累加（UPDATE）

UPDATE 状态分两拍完成（两段式，避免同一周期读旧写新）：

- 第 1 拍：算 L 与 8×4 个 weight $\times V$ 乘积并累加到 ACC[j]；
- 第 2 拍：把 S 个 ACC[j] 按维度求和进 ACC_reg（4 路）。

weight $\times V$ 采用 weight 高位补 0 成 6-bit 有符号后与 V 相乘，保证无符号 weight 与有符号 V 的乘积符号正确。

4.6 归一化（NORM）与四舍五入

为消除 $O = \text{ACC}/L$ 的截断误差，采用**按符号四舍五入**：正数加 $L/2$、负数减 $L/2$，再做向零截断的整除，等价于四舍五入（0.5 向远离 0 方向）：

$$O_i = \frac{\text{ACC}_i + \text{sign}(\text{ACC}_i) \cdot \lfloor L/2 \rfloor}{L}. \tag{6}$$

其中 $\lfloor L/2 \rfloor$ 由 L 逻辑右移 1 位（`L >>> 1`）得到。

4.7 输出（OUTPUT）

OUTPUT 状态用组合逻辑将 $O[cnt_out]$ 送出，out_valid 拉高，cnt_out 计数到 $d-1$ 后回到 IDLE 并置 $done = 1$。

5 加分项

5.1 C：参数化（S / d 可配）

模块头用 `parameter S=8, d=4` 声明，所有位宽与数组维度均由 localparam 从 S/d 用 $clog2 推导,改 S 或 d 只需改变几行代码。testbench 同步参数化并通过 `attention_ip #(.S(S),.d(d))` 传参，保证励与设计参数一致。

5.2 D：稀疏 Attention（动态阈值 Key 索引）

在 UPDATE 状态用二层循环 weight 求最大值 max_weight，以 `max_weight >> 1` 为动态阈值，判断 weight 数组中的每个数是否大于阈值，最后输出以 S 位的二进制数 key ，实现稀疏加速。

5.3 E：指数实现方法综述

指数运算其硬件实现主要有几种思路，对比如表 4：

表 4: 指数实现方法对比

方法	原理	精度 / 代价	适用场景
直接查表（LUT）	预存 e^x 离散值	单拍、面积最小，精度随表大小	输入为少量离散整数
分段线性	$y = a_i x + b_i$ 折线近似	单拍、中精度	连续输入、中等精度
多项式（Taylor/Padé）	乘加展开级数	高精度、多乘法器多拍	连续输入、高精度
CORDIC	旋转迭代求双曲函数	只需移位/加法、多拍	无乘法器、低面积
范围缩减 + 查表	$e^x = 2^{x \log_2 e}$，整数移位 + 小数查表	精度可调、面积中等	通用硬件指数

本题中 $\mathrm{diff}_j = \mathrm{score}_j - x_{\max}$ 是整数，且在二进制四舍五入下，只有四个数字有解，其余均四舍五入为 0。因此只需一张 5 行的 LUT，综合考虑下，采取 LUT 方案实现简单且高效。

5.4 G：$[256, 64]$ 规模 softmax 分析

若把规模放大到 $S = 256$、$d = 64$，softmax 的计算与存储开销显著增长，主要挑战如下：

存储： K、V 各 $256 \times 64 \times 8\,\mathrm{bit} = 16\,\mathrm{KB}$，合计 32 KB，超出小规模 FPGA 的片上寄存器/分布式 RAM，需改用 Block RAM 或外部存储。

位宽： $\mathrm{score} = Q \cdot K_j$ 为 64 项 8×8 点积，位宽 $16 + \lceil \log_2 64 \rceil = 22$ bit；$L = \sum \mathrm{weight}$ 最大 $256 \times 16 = 4096$，需 13 bit；ACC 累加 256 项，位宽进一步增大。

计算量： score 与 ACC 各需 256×64 次乘累加，共约 3.3×10^4 次 MAC。沿用本设计的串行状态机，仅 DOT 就需 16384 拍，必须引入流水线（加分项 B）或并行乘加阵列。

稀疏性： weight 随 diff 指数衰减，绝大多数 token 的 weight 为 0，S 越大越稀疏。加分项 D 的 key 掩码可跳过绝大多数 V 的乘加，计算量可降约一个数量级。

关键结论： exp LUT 与规模无关——无论 S/d 多大，diff 仍是整数，$e^{\mathrm{diff}} \times 16$ 仍只有 16/6/2/1/0 五档，5 行 LUT 依然成立。真正的瓶颈在存储带宽、MAC 数量与延迟，需靠分块、流水线与稀疏跳过协同解决。

6 验证与仿真

6.1 验证方法

采用「golden model 对照」策略：用 Python 实现与 RTL 完全一致的七步算法作为参考模型，testbench 通过 $\$readmemh$ 从十六进制激励文件读入 $Q/K/V$，驱动 RTL 运行，比较

out_data 序列与 golden 的 O。

6.2 仿真结果

六组用例下 RTL 的 *out_data* 序列均与 golden 的 O 完全一致，*done* 正确拉高，*key* 掩码与手工计算相符，验证了参数化、负数、四舍五入、稀疏阈值与大位宽等完整数据通路。

表 5: 仿真结果汇总

用例	规模	期望 O	*key*	结果
1（正数）	$S=8, d=4$	$[1,2,3,4]$	——	通过
2（负数）	$S=8, d=4$	$[-1,0,1,2]$	0000_0001	通过
3（参数化）	$S=10, d=5$	$[1,2,3,4,5]$	`0x200`	通过
4（负 0.5 边界）	$S=10, d=5$	$[-1,-1,-2,-2,-3]$	`0x3`	通过
5（全同 $L=128$）	$S=8, d=4$	$[1,2,3,4]$	`0xFF`	通过
6（大 score 稀疏）	$S=8, d=4$	$[11,24,-43,4]$	`0x04`	通过

6.3 周期数与时序

时钟由 `always #5 clk = ~clk` 产生，周期 10 ns（100 MHz）。一次完整计算（从载入 Q 到 *done* 拉高）各状态占用的时钟周期如表 6 所示。

表 6: 各状态周期数

状态	周期数	说明
LOAD_Q	4	连续 $d=4$ 拍载入 Q
LOAD_KV	32	$S \times d = 32$ 拍载入 K/V
DOT	32	8 个 score，每个 4 项点积
MAX	1	组合求 $x_{\max}$，一拍寄存
EXP	2	查表得 weight
UPDATE	3	两段式累加 L 与 ACC
NORM	2	除法与四舍五入
OUTPUT	4	逐拍输出 O 的 $d=4$ 个元素
合计	80	纯数据通路延迟

其中 EXP、UPDATE、NORM 因为用寄存的完成标志（EXP_finish 等）在下一拍才切换状态，各多占 1 拍。总计 80 个时钟周期，即 $80 \times 10\,\text{ns} = 800\,\text{ns}$；加上前端复位与启动握手约 35 ns，GTKWave 波形总长约 835 ns。

7 已知问题与改进方向

1. **除法可综合化**：NORM 中 $O = \text{temp}/L$ 的 `/` 目前是仿真友好写法（除数 L 为变量，不可综合），综合时需替换为**倒数 LUT**（$\frac{1}{L}$ 查表后乘 ACC）或定点除法器。

2. $L = 128$ **溢出**：L 最大 $S \times 16 = 128$，8-bit 有符号装不下 128。NORM 已用 `{1'b0,L}` 扩到 9-bit，并经测试用例 5（$L = 128$）验证正确。

3. **重复运行复位**：L、score_reg、ACC、ACC_reg 等中间寄存器仅由 rst_n 复位，第二次 *start*（不复位）会带旧值累加，需在进入新一轮时显式清零。